— Solution notes —

COMPUTER SCIENCE TRIPOS Part IB – 2024 – Paper 4

6 Programming in C and C++ (djg11)

Undefined behaviour

(*a*) A C programmer has an array of pointers to structs. They sort the array using the built-in comparison operator, '<'. Describe one circumstance where this will produce undefined behaviour and another situation where the behaviour is defined. [2 marks]

Answer: The C standard defines comparison using the built-in > operator for pointers to elements of the same array. Hence, where the structs have been allocated from an arena, the comparison will be defined and deterministic. If some pointers are null, comparison is undefined.

Further points (not needed): Where the structs have been allocated by separate calls to `new`, how they will compare is undefined and non-deterministic from run-to-run, but will be consistent when two pairs are compared more than once. Given typical segment structures, the comparison could be relied on to separate heap, stack and static instances, but this is not recommended, except perhaps for debugging certain corner cases!

had in mind. If pointers are signed, the arena could feasibly wrap the 2GByte boundary on an A32 machine, leading to unexpected (but deterministic) behaviour. Hence a cast to unsigned pointers may be needed on an A32 machine where they are signed by default.

[NB. A user-defined overload for the greater-than symbol cannot be classed as using the built-in operator.]

---

template classes

(*b*) Give three reasons why is it helpful to separate the declaration and implementation of classes in OO programming. The following code gives a (possibly incorrect) C++ signature declaration for a collection type. Criticise this definition and explain any difficulties that might arise with having the implementation in a separate file. If all types entered in the collection inherit from a common parent, can this make a difference? [8 marks]

```
template <typename T> class bucket
{ public:
  bucket(int N);     // Constructor: holds up to N items.
  int push(T *item); // Add item or return non zero if full.
  T pop();           // LIFO order pop most-recently added.
  T *dequeue();      // FIFO order dequeue of oldest item.
};
```

Answer: Implementation and definition should be usefully separated for reasons of IP protection, compilation speed and implementation independence (ie. good practice). *Marking schema: One mark for each reason, up to 3.*

The provided code has no private fields (so nowhere to store the contents except in undesirable globals) and another shortcoming of C++ is that the (implementation-dependent) size of these is needed at object instantiation time, so these also need to be defined in the declaration (sic)! Also, default constructor is not disabled and appropriate const annotations are missing.

However, C++ has a number of restrictions: template classes are expanded in-line by the compiler and hence the method bodies also need to be provided (although strictly they can be outside the class definition, still being compiled as though they were inside).

The inconsistent return types between `dequeue` and `pop` is undesirable. Also, if the queue is empty, returning a NULL reference is appropriate, or else a `count()` method should be supplied for the consumer to test. Ditto push when full. If the bucket is used for inter-thread message, blocking in the methods is a suitable alternative.

A common parent avoids the need for a templated implementation, hence separate compilation becomes possible. This can be in one of two forms. First, if signatures are adjusted so the bucket only stores references, these intrinsically all extend `void *` which is a common parent (Java-like type erasure). Storing plain-old-data such as integers is now not possible however. Alternatively, if `pop` returns a polymorphic value from inside the bucket (as per next section), all types stored in the `bucket` should inherit from a common parent that defines a copy constructor. This supertype can be hard-coded in the method signatures and again a template is no longer needed. See next answer. Of course, templating inlines code which can increase performance unless I-caches start overloading.

Values vs references

(*c*) Instead of holding pointers to objects, a collection class can hold the objects themselves. What changes in the `bucket` method types would be needed? What are the advantages and disadvantages of this change? [4 marks]

Answer: The asterisks can be removed from the `push`, `pop` and `dequeue` methods. NULL can no longer be used to report underrun errors so we are forced to use a `count` or `is_empty` method, or possibly an exception.

An advantage is that a copy of the object pushed is made, avoiding dangling reference problems or issues with the pushed item being externally mutated while in the `bucket`. The latter is a disadvantage if that behaviour was wanted! A disadvantage is copying overhead if the object is larger. Another disadvantage is polymorphic types cannot easily be stored as values based on only the information in a common supertype. *Marking schema: It is possible the first two points were answered already in part a. in which case credit vired here as appropriate. Two marks for any two further, relevant points.*

Dynamic polymorphism

(*d*) Returning to holding references in the collection, it is instead required that all objects pushed must have a `void foo()` method that the `bucket` will invoke for all members currently in the `bucket` on either removal operation. Define how items might be stored inside the `bucket` and sketch suitable code for the `dequeue()` method with this addition. Can the `foo()` method be invoked using static or dynamic method invocation? [6 marks]

Answer: A common supertype is needed, such as

```
class bucket_item { public: virtual void foo(); };
```

Here is one form of the code, but only the field definitions and the `dequeue` method are requested.

```
class bucket2
{
  int inptr=0, outptr=0, N, items=0;
  bucket_item **data;
public:
  bucket2(int N) : N(N) { data = new bucket_item* [N]; }
  int push(bucket_item *item)
  { if (items == N) return -1;
    data[inptr] = item; inptr = (1+inptr)%N; items+=1; return 0;
```

```
  }
  bucket_item *pop() // LIFO order pop
  {
    if (items == 0) return 0;
    for (int i=0, p=inptr; i<items;  i+=1, p=(p+1) %size) { data[p]->foo(); }
    inptr = (N-1+inptr)%N; bucket_item *r=data[inptr]; items-=1; return r;
  }
  bucket_item *dequeue() // FIFO order dequeue.
  { if (items == 0) return 0;
    for (int i=0, p=inptr; i<items;  i+=1, p=(p+1) %size) { data[p]->foo(); }
    bucket_item *r=data[outptr]; outptr = (1+outptr)%N; items-=1; return r;
  }
};
```

Nominal marking schema: 2 marks for superclass inference and definition, 2 marks for field definitions for an appropriate bounded data structure; 1 mark for the FIFO dequeue code, 1 mark for accurately invoking `foo()` *on all current entries.*

The keyword `virtual` used above made it dynamic. Removing the keyword gives static invocation which can be more efficient, but can only be used if no particular child type needs to override the `foo()` implementation. *Marking schema: One mark for explaining how it could be either. Maximum credit bounded at 6.*
